# Predicting Energy Consumption of Buildings

Using MongoDB, PyMongo and Machine Learning

---

**MongoDB Project Report**

Submitted to: Dr. Ashu

Student Name: _____

Roll No. / Section: _____

Date: October 2026

# 1. Introduction

Buildings account for a large share of global electricity use, and heating, cooling, lighting and occupant activity are the main contributors. Being able to predict how much energy a building will consume helps facility managers plan capacity, detect abnormal usage, schedule maintenance and reduce cost and emissions.

Energy-monitoring systems produce large volumes of semi-structured readings (sensor values, weather feeds, occupancy counts) with attributes that vary between buildings. A document database such as MongoDB is a natural fit: each reading is stored as a flexible JSON-like document, new fields can be added without schema migration, and the aggregation framework allows quick exploratory analysis directly inside the database.

This project builds an end-to-end pipeline in which energy readings are stored and queried in MongoDB through the PyMongo driver, pulled into Python for preprocessing, and used to train and compare machine learning models (Linear Regression, Logistic Regression, Decision Tree and Random Forest).

# 2. Project Objective

The main objective is to predict the energy consumption of buildings from weather conditions, occupancy and building characteristics, using MongoDB as the data store. The specific goals are:

- Store an energy-consumption dataset (with weather and occupancy data) in MongoDB using PyMongo, and query it with filters and aggregation pipelines.
- Clean the data (missing values, duplicates) and encode categorical variables such as building type, weather condition and season.
- **Regression task:** predict the exact energy consumption in kWh and evaluate with $R^2$ Score and Mean Absolute Error (MAE).
- **Classification task:** predict whether a reading is High or Normal usage and evaluate with Accuracy, Precision, Recall and F1-score.
- Compare several models and identify the best performer and the most influential factors.

# 3. Dataset Description

The dataset contains hourly-timestamped energy readings for 24 buildings of five types (Office, Residential, Retail, School and Hospital) during the year 2025, with 250 readings per building. Because no real metering data was supplied with the assignment, the dataset was generated by the script generate_dataset.py (fixed random seed 42, so it is fully reproducible). It is therefore a synthetic dataset: the relationships in it are realistic by construction (for example, HVAC load grows as the outdoor temperature moves away from a 21 °C comfort point, and occupancy adds load), but the numbers are not measurements from real buildings. The same pipeline can be applied to a real dataset with the same columns.

The raw file has 6,015 records and 14 columns. About 2% of the values of temperature, humidity and occupancy were deliberately left missing, and 15 duplicate records were added, so that the preprocessing stage has genuine cleaning work to do.

| Column | Type | Description |
|---|---|---|
| **record_id** | Integer | Unique reading identifier |
| **building_id** | String | Building code (B001–B024) |
| **building_type** | Categorical | Office, Residential, Retail, School, Hospital |
| **floor_area** | Numeric | Floor area in m² (1,531–8,967) |
| **year_built** | Integer | Construction year (1964–2020) |
| **timestamp** | Datetime | Date and hour of the reading |
| **temperature** | Numeric | Outdoor temperature in °C |
| **humidity** | Numeric | Relative humidity in % |
| **wind_speed** | Numeric | Wind speed in m/s |
| **solar_radiation** | Numeric | Solar radiation in W/m² |
| **weather_condition** | Categorical | Clear, Cloudy, Rainy, Foggy |
| **occupancy** | Numeric | Number of people in the building |
| **is_weekend** | Binary | 1 for Saturday/Sunday, otherwise 0 |
| **energy_kwh** | Numeric (target) | Energy consumed in the hour, in kWh (mean 675, range 44–2,520) |

Figure 1 and Figure 2 give a first look at the data: consumption differs clearly between building types, and follows a daily cycle with a peak in the early afternoon.

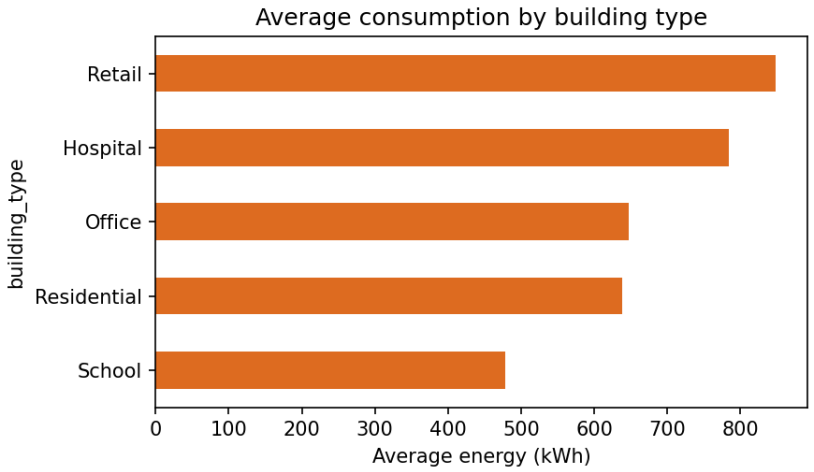



*Figure 1: Average energy consumption by building type*

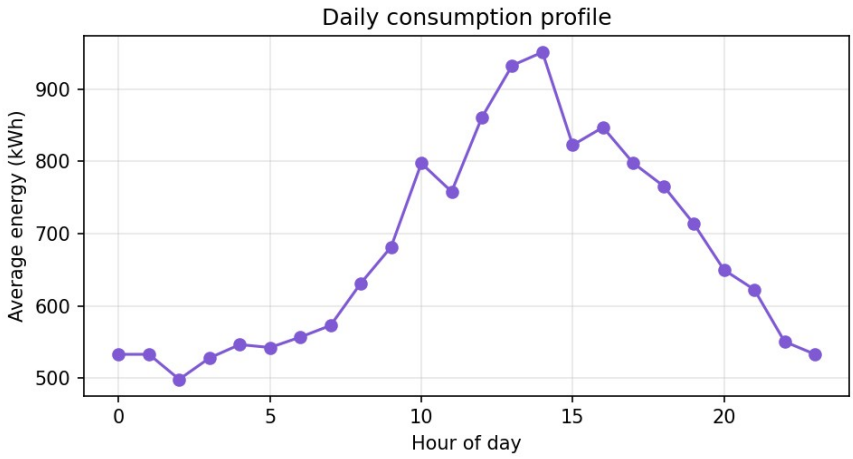



*Figure 2: Average energy consumption by hour of day*

# 4. MongoDB / PyMongo Implementation

## 4.1 Database design

A database named energy_db is used with two collections: energy_readings (one document per reading, 6,015 documents) and model_results (one document per trained model with its evaluation metrics). Each reading is stored as a flat document. Three derived fields (hour, month, day_of_week) are added at load time so that aggregations can group on them directly. Missing values are stored as null.

```
{
  "record_id": 3684, "building_id": "B015", "building_type": "Hospital",
  "floor_area": 4238.0, "year_built": 1982,
  "timestamp": ISODate("2025-01-01T01:00:00Z"),
  "temperature": 10.36, "humidity": 78.5, "wind_speed": 7.5,
  "solar_radiation": 0.0, "weather_condition": "Cloudy",
  "occupancy": 185.0, "is_weekend": 0, "energy_kwh": 844.81,
  "hour": 1, "month": 1, "day_of_week": 2
}
```

## 4.2 Connection and data loading

The connection is created in mongo_utils.py with MongoClient and verified with a ping command. The CSV is read with pandas, NaN values are converted to None (stored as null), NumPy types are converted to native Python types, and all documents are inserted in one call with insert_many(). Indexes on building_id, timestamp and building_type speed up filtering and grouping.

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017", serverSelectionTimeoutMS=2000)
client.admin.command("ping")
col = client["energy_db"]["energy_readings"]

col.delete_many({})                # makes the script re-runnable
col.insert_many(records)            # 6,015 documents
col.create_index([("building_id", 1)])
col.create_index([("timestamp", 1)])
```

```
col.create_index([("building_type", 1)])
```

### 4.3 Queries and aggregation pipelines

Several queries were run to explore the data inside MongoDB before any machine learning. Simple counts with filters were used for data-quality checks, and the aggregation framework ($group, $sort, $limit) was used for summaries.

```
col.count_documents({})                              # total documents
col.count_documents({"temperature": None})           # missing temperature
col.count_documents({"energy_kwh": {"$gt": 1500}})# very high usage

col.aggregate([
  {"$group": {"_id": "$building_type",
              "avg_kwh": {"$avg": "$energy_kwh"},
              "readings": {"$sum": 1}}},
  {"$sort": {"avg_kwh": -1}}])
```

| Query | Result |
|---|---|
| **Total documents** | 6,015 |
| **Documents with missing temperature** | 120 |
| **Weekend readings (is_weekend = 1)** | 1,743 |
| **Readings above 1,500 kWh** | 201 |
| **Highest average by building type** | Retail (848 kWh), then Hospital (784), Office (649), Residential (639), School (478) |
| **Average by weather condition** | Clear 833, Cloudy 652, Rainy 617, Foggy 584 kWh |
| **Peak hours (average kWh)** | 14:00 (948), 13:00 (933), 12:00 (861), 16:00 (847), 15:00 (821) |
| **Highest total consumption** | Building B023 (341,712 kWh), then B020, B015, B006, B011 |

After training, the evaluation metrics of every model are written back to the model_results collection with insert_many(), so that both the data and the results live in MongoDB.

## 5. Data Preprocessing

The training script reads all documents back from MongoDB with find({}, {"_id": 0}) and converts them to a pandas DataFrame. The following steps were then applied:

- **Duplicate removal:** 15 duplicate records (same record_id) were dropped, leaving 6,000 unique readings.
- **Missing values:** temperature (120), humidity (121) and occupancy (120) had missing entries. Temperature was filled with the median of its month; the remaining numeric gaps were filled with the column median using SimpleImputer inside the model pipeline, so the medians are learned from the training set only.
- **Feature engineering:** building_age = 2025 − year_built; temp_deviation = |temperature − 21| (distance from the comfort temperature, which drives heating/cooling load); season

derived from the month (Winter, Spring, Summer, Autumn); hour and month from the timestamp.

- **Categorical encoding:** building_type, weather_condition and season were converted to numeric columns with one-hot encoding (OneHotEncoder, unknown categories ignored).
- **Scaling:** numeric features were standardised with StandardScaler for Linear and Logistic Regression. Tree-based models do not need scaling.
- **Classification target:** a reading is labelled High usage if its consumption is above the 66.7th percentile (823.19 kWh), otherwise Normal. This gives roughly one third High and two thirds Normal readings.
- **Train/test split:** 80% training (4,800 readings) and 20% testing (1,200 readings) with random_state = 42; the classification split is stratified. The encoder, scaler and imputer are part of a scikit-learn Pipeline, so no information from the test set leaks into training.

# 6. Model Implementation

Two prediction tasks were defined on the same set of 14 input features (11 numeric and 3 categorical). Models were implemented with scikit-learn.

## 6.1 Regression: predicting energy_kwh

- **Linear Regression** (baseline): models consumption as a weighted sum of the standardised and encoded features.
- **Random Forest Regressor:** an ensemble of 300 decision trees (min_samples_leaf = 2) that averages their predictions and can capture non-linear effects and interactions, such as the dependence of consumption on both building type and time of day.

## 6.2 Classification: High vs Normal usage

- **Logistic Regression:** linear classifier with standardised inputs (max_iter = 2000).
- **Decision Tree:** single tree with max_depth = 8 and min_samples_leaf = 5 to limit over-fitting; easy to interpret.
- **Random Forest Classifier:** 300 trees, min_samples_leaf = 2.

```
pipe = Pipeline([("pre", make_preprocessor(scale=False)),
                 ("m", RandomForestRegressor(n_estimators=300,
                        min_samples_leaf=2, random_state=42))])
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

# 7. Evaluation Results

## 7.1 Regression results

$R^2$ measures the proportion of variance in energy consumption explained by the model (1.0 is perfect). MAE is the average absolute error in kWh; RMSE is also reported because it penalises large errors more strongly.

| Model | $R^2$ Score | MAE (kWh) | RMSE (kWh) |
|---|---|---|---|
| **Linear Regression** | 0.9123 | 92.65 | 120.93 |
| **Random Forest Regressor** | 0.9578 | 56.26 | 83.86 |

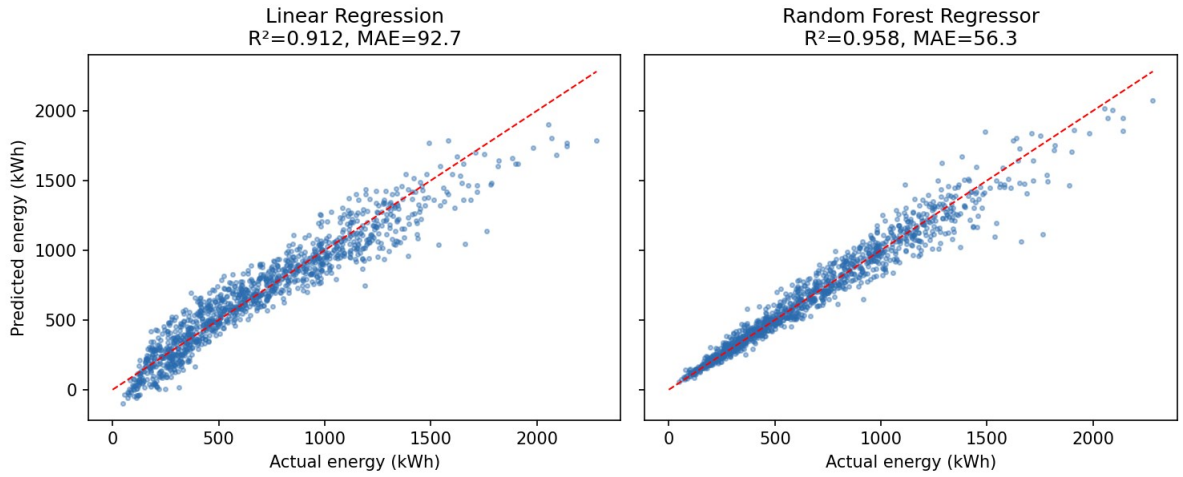



*Figure 3: Actual vs predicted energy consumption on the test set (dashed line = perfect prediction)*

The Random Forest Regressor clearly outperforms the Linear Regression baseline: $R^2$ rises from 0.9123 to 0.9578 and the average error falls from 92.65 kWh to 56.26 kWh (about 39% lower). The linear model is already strong because consumption depends strongly on floor area and occupancy, but it cannot represent the non-linear temperature effect or the differing daily schedules of building types.

## 7.2 Classification results

| Model | Accuracy | Precision | Recall | F1-score |
|---|---|---|---|---|
| **Logistic Regression** | 0.9433 | 0.9213 | 0.9075 | 0.9144 |
| **Decision Tree** | 0.9275 | 0.8808 | 0.9050 | 0.8927 |
| **Random Forest Classifier** | 0.9550 | 0.9303 | 0.9350 | 0.9327 |

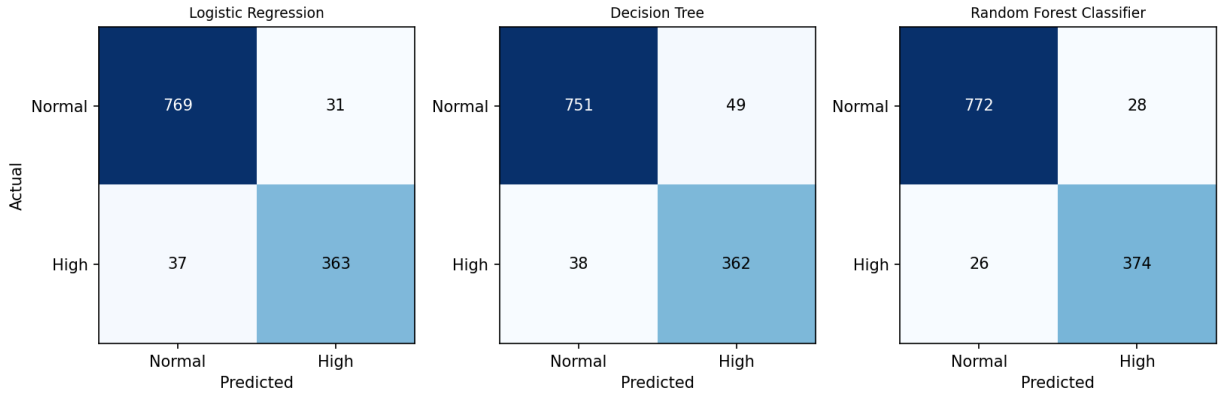



*Figure 4: Confusion matrices on the test set (1,200 readings; High usage is the positive class)*

The Random Forest Classifier gives the best results on every metric (accuracy 0.9550, F1 0.9327). Logistic Regression is a close second, and the single Decision Tree is the weakest of the three but still

above 92% accuracy. Precision and recall are well balanced for all models, meaning that the models neither raise many false alarms nor miss many truly high-consumption readings.

### 7.3 Most influential features

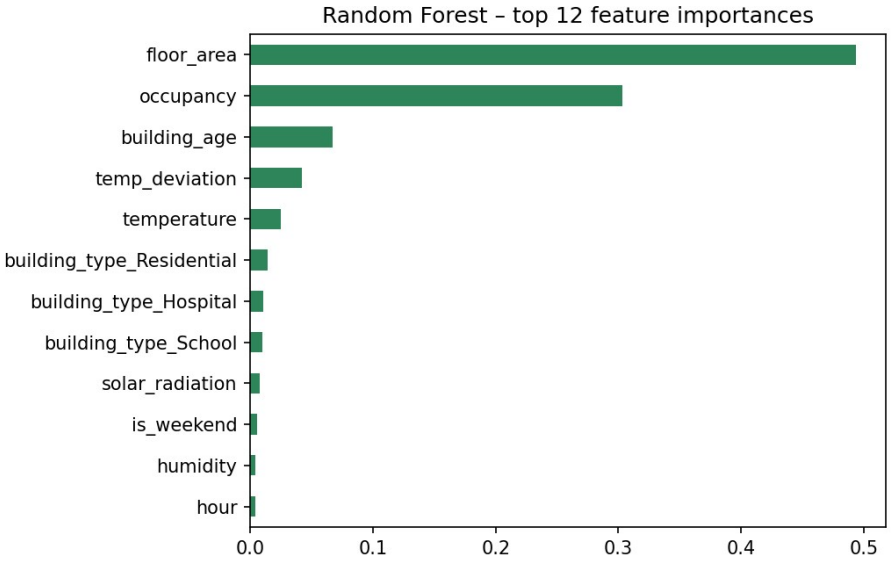



*Figure 5: Feature importance from the Random Forest Regressor*

Floor area (about 49% of the importance) and occupancy (about 30%) are by far the most important predictors, followed by building age (7%) and the distance of the temperature from the comfort point (4%). This agrees with the intuition that larger and busier buildings use more energy, and shows that the engineered temp_deviation feature is more useful than raw temperature alone.

## 8. Conclusion

This project showed how MongoDB and PyMongo can be used as the data layer of a machine learning workflow. Building energy readings were stored as documents, indexed, explored with filters and aggregation pipelines, read back for preprocessing, and the final model metrics were stored in MongoDB again.

Among the models tested, the Random Forest performed best for both tasks: as a regressor it reached $R^2$ = 0.9578 with MAE = 56.26 kWh, and as a classifier of High vs Normal usage it reached 95.5% accuracy with precision 0.9303, recall 0.9350 and F1 0.9327. Floor area and occupancy are the dominant drivers of consumption.

### Limitations and future work

- The dataset is synthetic, so the very high scores should be read as a demonstration of the pipeline rather than as expected performance on real meter data; real data is noisier. Repeating the study on a real dataset (for example a public building-energy benchmark) is the natural next step.
- Readings are sampled at random times, so the models treat each reading independently. Time-series features (previous-hour consumption, rolling averages) and a time-based train/test split would suit forecasting use.

- Hyper-parameter tuning with cross-validation, gradient-boosting models, and storing trained models and predictions in MongoDB could further improve and operationalise the system.

## Appendix: How to Run the Project

```
pip install -r requirements.txt
# start MongoDB (mongod) on localhost:27017
python run_all.py        # generate data, load into MongoDB, train, evaluate
```

Output files (metrics, query results and figures) are written to the outputs/ folder. The source files are: generate_dataset.py, mongo_utils.py, load_data.py, train_models.py, run_all.py and mini_mongo.py.